Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow
Apr 25, 2020 · 7 min read · ✦ · Listen

Save

HTTPS://TOWARDSDATASCIENCE.COM/TAGGED/SPACE-SCIENCE-WITH-PYTHON

Space Science with Python — A look at Kepler's first law

Second part of the Python tutorial series

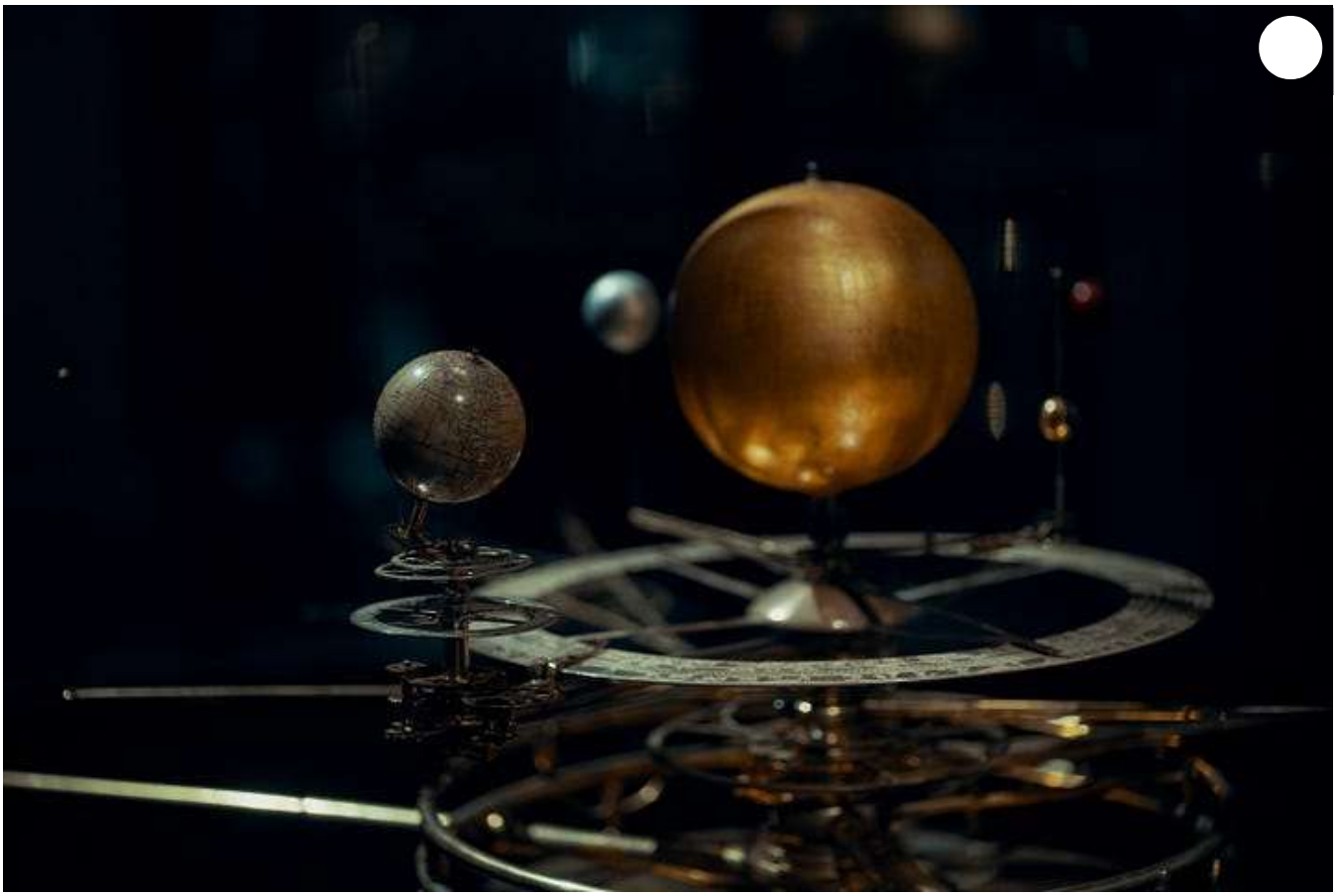


Photo by Ross Sneddon on Unsplash

94

Preface

This is the 2nd part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

Introduction

> *The orbit of a planet is an ellipse with the Sun at one of the two foci*

This is the first (of three) Kepler's laws of planetary motion. 400 years ago Johannes Kepler derived the laws based on long term observations from his and Tycho Brahe's research. He used a huge amount of manually determined planetary position data that lead him to his conclusions. Imagine being a data scientist with tons of printed Excel sheets, no computer, some mechanical mathematical devices and some patience. No GPU optimization or Cloud Computing. This was a true commitment to science.

But is the Sun *really* and *exclusively* in the centre of our Solar System? Considering Newton's law of mechanic our home planet, asteroids, planets and also the Sun revolve around a common centre of gravity. The Sun contains over 99 % of the Solar System's mass. So is a possible movement of the so-called Solar System Barycentre (SSB) with respect to the Sun's centre *that* significant? We will answer this question today with our Python skills and our SPICE knowledge from the first tutorial.

Preparation

Last time we learned about SPICE and its way to compute miscellaneous parameters using so called kernels. We loaded the necessary kernels using the SPICE function furnsh. Each kernel was loaded individually. However, larger projects may require to load dozens of kernels. That would bloat our Python code and one could easily lose track of one's code. To prevent this, SPICE introduced the kernel meta files. A meta file contains the relative or absolute paths to the needed kernels. For this tutorial, each part has its own meta file that sets the relative path to the *_kernels* directory. Let's have a look at the meta file:

kernel_meta.txt — The SPICE kernel meta file for this tutorial

The first 6 lines start with the command *\begintext*. The following text block is a comment section, where one can add up any project relevant information. The last block starts with *\begindata*. This indicates SPICE that all following parts are relevant for the coding part. *KERNELS_TO_LOAD* lists all kernel files that are used in this tutorial part. Please note: If you load kernels that contain similar information (e.g., by accident you load first the naif0012.tls leap seconds kernel and afterwards the naif0011.tls kernel), SPICE considers the order and takes the last one.

Let's start

Now we are prepared for our project. First of all, we need to import the necessary modules. If you have not installed *NumPy* yet, please install it via *pip3*. We will need it frequently. Then function *furnsh* is used to load our newly created kernel meta file.

Part 1/8

For the position computation we need to set a starting / initial and ending time-stamp. Here, we set the date 2000–01–01 (midnight) UTC and add up 10,000 days (around 27.5 years; this value is chosen randomly by me). These computations can be done with the Python library *datetime*. Afterwards, the initial and ending times are converted to strings in line 14 and 15, respectively. Finally, the UTC strings are converted to the corresponding Ephemeris times.

Part 1/8

Why are leap-seconds kernels necessary? A day has 24 hours, a year has 365 days and every 4 years another day is added. This applies to our everyday life. From a

detailed scientific point of view our Earth revolves around itself and around the Sun in slightly varying times that need to be compensated. How much time is added in this 10,000 days interval starting at the millennium?

A day has 86,400 seconds. This value times 10,000 days leads to 864,000,000 seconds. Subtracting the initial time in ET from the ending time in ET (line 5) leads to 864,000,005.0012845 seconds. Approximately 5 seconds have been added. This is not crucial for our tutorial, but in some high precision scientific fields every millisecond counts.

We finally set a NumPy array that contains 10,000 time steps between the starting and ending times.

Part 3/8

Now we are ready to compute the Solar System Barycentre (SSB) position in x, y, z direction with respect to the Sun's centre. For this purpose, we compute the position using the SPICE function *spkgps* (in the last tutorial we used a similar function *spkgeo* that returns the state vector (position and velocity)). This function requires the following input parameters:

- *targ*: The NAIF ID code of the SSB (0). The sophisticated SPICE documentation lists all available ID codes of the Solar System as well as spacecraft missions
- *et*: The Ephemeris time
- *ref*: The reference frame of interest. In our case, we compute the position in ECLIPJ2000. So, the ecliptic plane of our home planet is the reference plane
- *obs*: The NAIF ID of the Sun (10)

spkgps returns the position of the SSB w.r.t. the Sun at a certain Ephemeris time computed in the ECLIPJ2000 reference frame. The second output value is the so called light time and stores the travelling time of the light between the Sun's centre and the SSB. Since we do not need this value, we use a single underscore.

Part 4/8

Let's print the result at the initial time point to get a feeling of the x, y, z components and the distance.

Part 5/8

We get the following output:

```
Position (components) of the Solar System Barycentre w.r.t the
centre of the Sun (at initial time):
X = 1068000.0 km
Y = 417681.0 km
Z = -30845.0 km
```

```
Distance between the Solar System Barycentre w.r.t the
centre of the Sun (at initial time):
d = 1147185.0 km
```

Well … we got some large numbers. A distance between the Sun's centre and the SSB of over 1 Million kilometres (around 3 times the distance between the Earth and the Moon). Is this … a lot?

Comprehending large numbers in astronomy leads mostly to scaling, like e.g., 1 Astronomical Unit (AU) or 1 Light-year. Here, we can define our own scale, by using the radius of the Sun (around 700,000 km). The Sun's radius can be extracted from the corresponding SPICE kernel.

Before we go into the programming part we need to answer another question. *What is the radius of the Sun?* It is somehow connected to the surface, but *what is the surface of a huge plasma ball*?

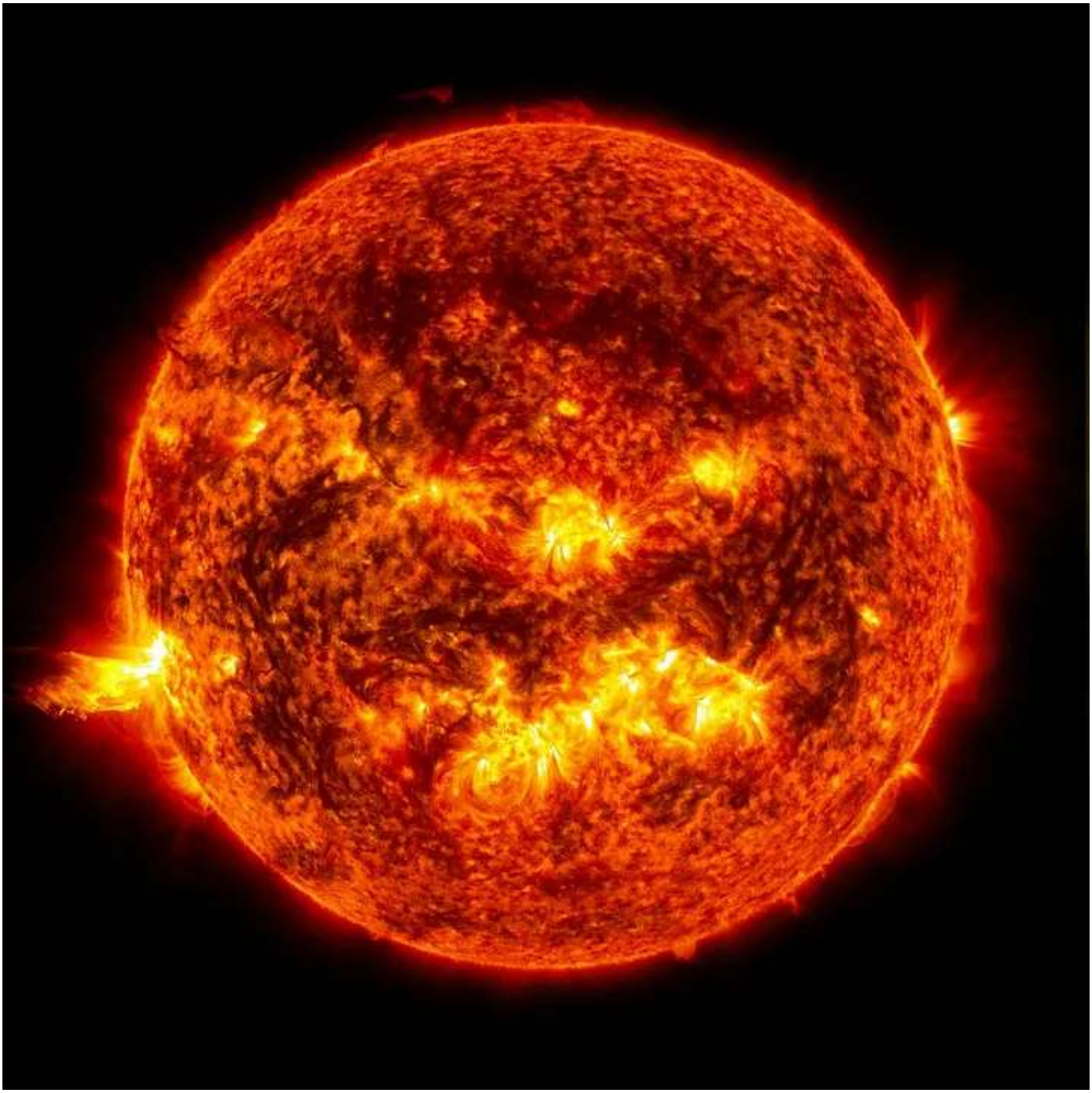


Image of the Sun from NASA's Solar Dynamics Observatory (SDO). Credit: Credit: NASA/Goddard/SDO

The answer is complex. Different methods are available for the definition of the Sun's surface. Helioseismological measurements, time measurements of mercury transits, or coupling the definition of the surface with a certain layer of the Photosphere (where one can observe the convection cells and sunspots) lead to different results. In 2015, the International Astronomical Union set the radius to 695,700 km, period. A slightly shifted value (696,000 km) is stored in the SPICE kernel *pck00010.tpc* that is loaded in the beginning of our tutorial.

With the function *bodvcd* we can extract the radius of the Sun. The function requires 3 parameters:

- *bodyid*: The NAIF ID of the Sun (10)
- *item*: 'RADII' tells the function that we need the radius information of the Sun
- *maxn*: The number of expected return parameters. Since we get the radius values of an ellipsoid (x, y, z direction) the number is set to 3.

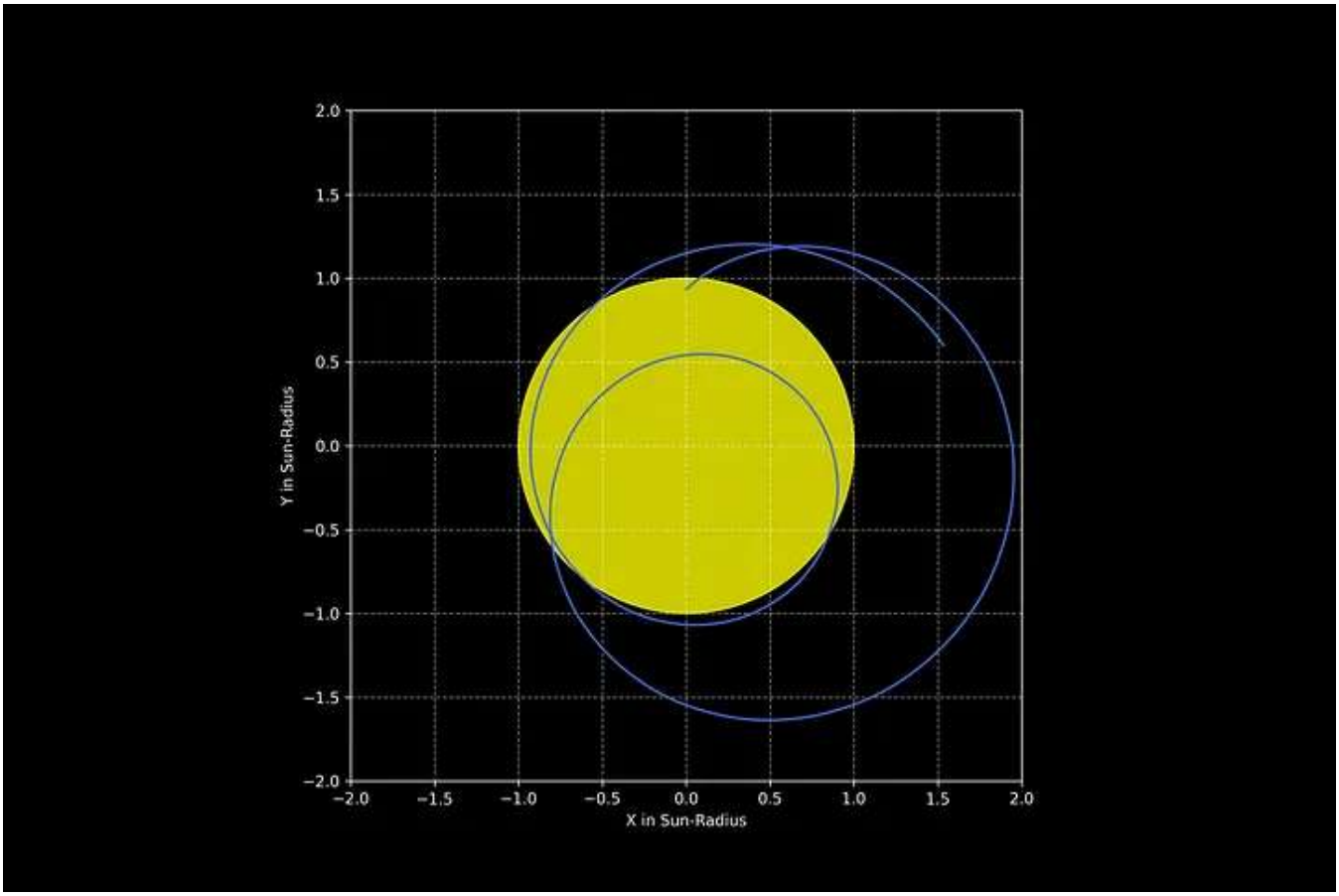
The first output parameter represents the number of returned values and is not needed. We use a single underscore to ignore this value. Since all radii values are the same, we use the first entry as the Sun's radius.

Part 6/8

Now, we can plot the Sun and the trajectory of the SSB. We use *matplotlib* for this task. We create a 2-dimensional projection of our results and plot only the x-y coordinates (movement on the Ecliptic plane).

The following code snippet shows how the plot below is created. We add a yellow circle representing the Sun. Since the distances are scaled w.r.t. the Sun's radius we can simply use a radius of 1.

Part 7/8



Trajectory of the Solar System Barycentre (blue) w.r.t the centre of the Sun (yellow). The plot shows a projection on the x-y plane (Ecliptic plane). 1 Sun-Radius corresponds to 696,000 km.

The plot shows the trajectory in blue and one can easily see, that the SSB is not within the Sun all the time (although the Sun contains over 99 % of the Solar System's mass)!

What is the percentage of time the SSB stays outside of the Sun? Let's have a look:

In total: around 65%! An astonishing result.

Conclusion

I hope you enjoyed today's lesson. We used some already known SPICE functions from the last tutorial and have deepen our knowledge. We will continue with that and steadily we will build up a "knowledge fundament" for more complex tasks.

So answering the question from the beginning: *Is the Sun in the centre in one of the foci of the orbits?* Well it is the barycentre of the Solar System. The Sun is "wobbling" around this centre. Does it mean the Kepler laws are wrong? Well that's science: You postulate a theory that describes the nature until it is proven wrong or it is improved. For no-high-precision measurements and computer models the Sun can be easily put in the centre of our Solar System.

The next tutorial will be published on 29th April 2020. Stay tuned and have a great weekend,

Thomas

Python Science Space Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play